

Project REPORT

SE2052 – Programming Paradigms



IT24101503

A. A. B. Shamim Basha

Y2 S2 – BSc (Hons) in Computer Science

03rd of May 2026

Contents

PROBLEM STATEMENT..... 3

TASK 1: SCOPE 3

TASK 2: LANGUAGE DESIGN AND THE FORMAL GRAMMAR 5

SAMPLE PROGRAMS..... 8

REFLECTION11

TaskLang++

A Domain-Specific Language for Task Scheduling and Automation

Problem Statement

Modern systems rely heavily on task scheduling and automation, from simple reminders to complex workflows (e.g., CI/CD pipelines, cron jobs, smart assistants). General-purpose languages can express these, but they are often verbose and error prone.

In this assignment, I have designed a Domain-Specific Language (DSL) called TaskLang++ to define:

- Tasks
- Time-based scheduling
- Dependencies between tasks
- Conditional execution

I have formally defined the syntax using BNF and EBNF and demonstrate how my DSL simplifies task specification. I have implemented a lexer using Flex and a parser using Bison.

Task 1: Scope

TaskLang++ is a DSL that is written in a declarative style and aims to provide a syntax for specifying automated tasks that involve scheduling based on time, dependency between tasks, and conditional execution. It is aimed at system administrators, DevOps engineers, and automation developers.

Types of task supported:

The DSL is specifically designed for the execution of scripts in mind as the main objective, where any external program or script can be executed. This ensures simplicity of the language while still maintaining its extensibility.

Scheduling Mechanism:

- Runs once at a specified time.
- Runs daily at the same time.
- Run weekly on a specific day
- Runs once.

Dependencies:

- After a specific task completes.
- Before starting a specific task.

Conditions:

- Execute only if the depended-upon task succeeded.
- Execute only if the depended-upon task fails

Constraints:

- **Unique tasks** – No duplicate task definition.
- **Acyclic dependency** - There should be no cycles in the dependency.
- **Time format** – Times are in HH:MM format (hours 00-23; minutes 00-59)
- **Filename** – Filenames need to have extensions (e.g., .py, .sh)

Assumptions:

- knows how to execute the provided program/scripts.
- Times are provided in 24-hour format.
- Times provided are in same time-zone.
- Input files is a valid executable script/program.

Task 2: Language Design and the Formal Grammar

BNF

<program>	::= <tasks>
<tasks>	::= <task> <task> <tasks>
<task>	::= "TASK" <identifier> "{" <run_command> <command> "}"
<command>	::= <schedule_command> <dependency_command>
<run_command>	::= "RUN" <filename>
<schedule_command>	::= "AT" <time> "EVERY" "DAY" "AT" <time> "EVERY" "WEEK" "ON" <day_of_week> "AT" <time>
<day_of_week>	::= "MONDAY" "TUESDAY" "WEDNESDAY" "THURSDAY" "FRIDAY" "SATURDAY" "SUNDAY"
<dependency_command>	::= "AFTER" <identifier> <opt_conditional_command> "BEFORE" <identifier>
<opt_conditional_command>	::= "IF" <condition> ε
<condition>	::= "success" "failure"
<filename>	::= "" <identifier> "." <ext> ""
<ext>	::= <char> <char><ext>
<identifier>	::= <char> <id_tail>
<id_tail>	::= <char> <id_tail> <digit> <id_tail> ε
<char>	::= a b c d e f g h i j k l m n o p q r s t u v w x y z A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
<time>	::= <digit><digit> ":" <digit><digit>
<digit>	::= 0 1 2 3 4 5 6 7 8 9

EBNF

<program>	::= <task> {<task>}
<task>	::= "TASK" <identifier> "{" <run_command> <command> "}"
<command>	::= <schedule_command> <dependency_command>
<run_command>	::= "RUN" <filename>
<schedule_command>	::= "AT" <time> "EVERY" "DAY" "AT" <time> "EVERY" "WEEK" "ON" <day_of_week> "AT" <time>
<day_of_week>	::= "MONDAY" "TUESDAY" "WEDNESDAY" "THURSDAY" "FRIDAY" "SATURDAY" "SUNDAY"
<dependency_command>	::= "AFTER" <identifier> [<conditional_command>] "BEFORE" <identifier>
<conditional_command>	::= "IF" <condition>
<condition>	::= "success" "failure"
<filename>	::= "'" <identifier> "." <ext> "'"
<ext>	::= <char>{<char>}
<identifier>	::= <char> (<char> <digit>)*
<char>	::= a b c d e f g h i j k l m n o p q r s t u v w x y z A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
<time>	::= <digit><digit> ":" <digit><digit>
<digit>	::= 0 1 2 3 4 5 6 7 8 9

Following are some tokens that can be used in TaskLang++

Token	Description
Task	Begins a task definition
RUN	Specifies command execution
EVERY	Recurring schedule keyword
DAY	Used with EVERY to execute daily
WEEK	Used with EVERY to execute weekly
AT	Time-based execution
ON	Specifies the day of the week
AFTER	Dependency specification for executing task after completion
Before	Dependency specification for executing task before execution
IF	Conditional execution
Success	Condition keyword for True case
Failure	Condition keyword for False case
HH:MM	Specifies the time in 24-hour format
Filename.ext	Specifies the filename to parse
IDENTIFIER	Specifies the unique identifier of tasks defined

Sample Programs

Valid Programs

One-Time Scheduled Tasks

	Output
<pre>TASK dailyReport { RUN "report.py" }</pre>	<pre>Parsing TaskLang++ input... --- EXECUTION START --- Executing Task: dailyReport Script: "report.py" --- EXECUTION COMPLETE ---</pre>

Simple Daily Scheduled Task

	Output
<pre>TASK dailyReport { RUN "report.py" EVERY DAY AT 06:00 }</pre>	<pre>Parsing TaskLang++ input... --- EXECUTION START --- Executing Task: dailyReport Script: "report.py" Schedule: EVERY DAY AT 06:00 --- EXECUTION COMPLETE ---</pre>

Multi-Step Workflow with Dependencies and Conditions

	Output
<pre>TASK backupDB { RUN "backup.sh" EVERY DAY AT 02:00 }</pre>	Parsing TaskLang++ input... --- EXECUTION START --- Executing Task: backupDB Script: "backup.sh" Schedule: EVERY DAY AT 02:00
<pre>TASK sendReport { RUN "report.py" AFTER backupDB IF SUCCESS }</pre>	Executing Task: sendReport Script: "report.py" Depends on: backupDB Runs after backupDB Condition: success
<pre>TASK cleanup { RUN "cleanup.sh" EVERY WEEK ON SUNDAY AT 03:00 }</pre>	Executing Task: cleanup Script: "cleanup.sh" Schedule: EVERY WEEK ON SUNDAY AT 03:00 --- EXECUTION COMPLETE ---

Invalid programs

Duplicate Task names

[illegible]

Circular Dependency

[illegible]

Reflection

Challenges Faced

- **Scheduling ambiguity** – If the task have both schedule command and dependency command, what should the task give the priority to execute?
- **Duplicate Task definition** – How to identify if the task is defined multiple times?
- **Storing dependency** – How to store the dependency between tasks?
- **Cyclic dependency detection** – How to identify if cycle exist between tasks dependency?
- **Forward References in Dependencies** – What to do if task is dependent upon a task that is not defined yet, How we connect those tasks dependency?
- **Time Validation** – How to verify the time scheduled is valid?

Solution

- **Scheduling ambiguity** - Each Task have only either schedule command or dependency command, So that there wouldn't be any ambiguity.
Trade-off – Task can have only one commad.
- **Duplicate Task definition** - Stores each task defined in an array, So that we can know what are the task defined and not defined
Trade-off – More memory used and only specific number of tasks can be defined
- **Storing dependency** - Keeps a adjacency matrix to store the nodes(tasks) and edges(dependency) of the tasks defined.
Trade-off – More memory used.
- **Cyclic dependency detection** – Before adding the dependency edge to the adjacency matrix, we check if there is a path to go from taskA to taskB. If there is a path exists, there exists a cycle so we stops parsing and print the error message.
Trade-off – More computation.
Takes time

- **Forward References in Dependencies** – We keep one array to store all the task and another array to store if that array is yet defined or not, so that we can just add the forward referenced task in the tasks array but keep the task_defined array for that task to be false. Preventing that the parser to not think that the task we have seen is defined and it is a duplicate when that task is being defined.

Trade-off – More Memory used.

- **Time Validation** – We validate the time in the parsing , not in the lexical analysis.

Trade-off – Program may pass lexical analysis but fails in Parsing.

Improvements

- **Parameter passing** - Allow data passing between dependent tasks.
- **Retry mechanism** - Add RETRY mechanism if any task fails.
- **Recurring tasks** - Runs 3 TIMES EVERY 5 MINUTES.
- **Parallel execution** – Ability to run tasks simultaneously.
- **Time zone support** - Add TIMEZONE for time scheduled tasks.

Conclusion

In conclusion, I have designed a Domain-Specific Language (DSL) called TaskLang++ to define:

- Tasks
- Time-based scheduling
- Dependencies between tasks
- Conditional execution

I have formally defined the syntax using BNF and EBNF and demonstrated how my DSL simplifies task specification and I have implemented a lexer using Flex and a parser using Bison.

In this project, I have gained hands on experience in language design using lexer and parser and learned how to convert the BNF/EBNF grammar rules into lexer and parser and the integration of lexer and parser to work seamlessly.